Primary Keys and Foreign Keys

Lecture Notes · Game DB Junior

In this lesson

- What a **primary key** is
- What a **foreign key** is
- How they build a **one-to-many / many-to-one** relationship

1. From relationships to data

We know the four kinds of relationships. Now we need to show them in **data**: how do we set up a relationship inside a table? For this we learn two new words: **primary key** and **foreign key**. They are extra columns that we add to many tables.

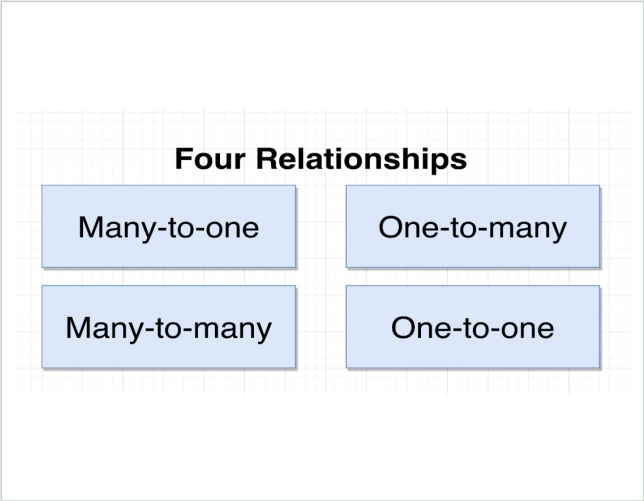


Figure 1 — The four relationships

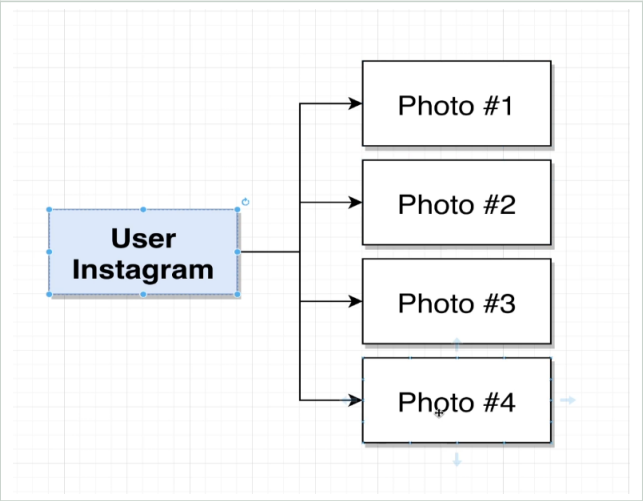


Figure 2 — A user has many photos

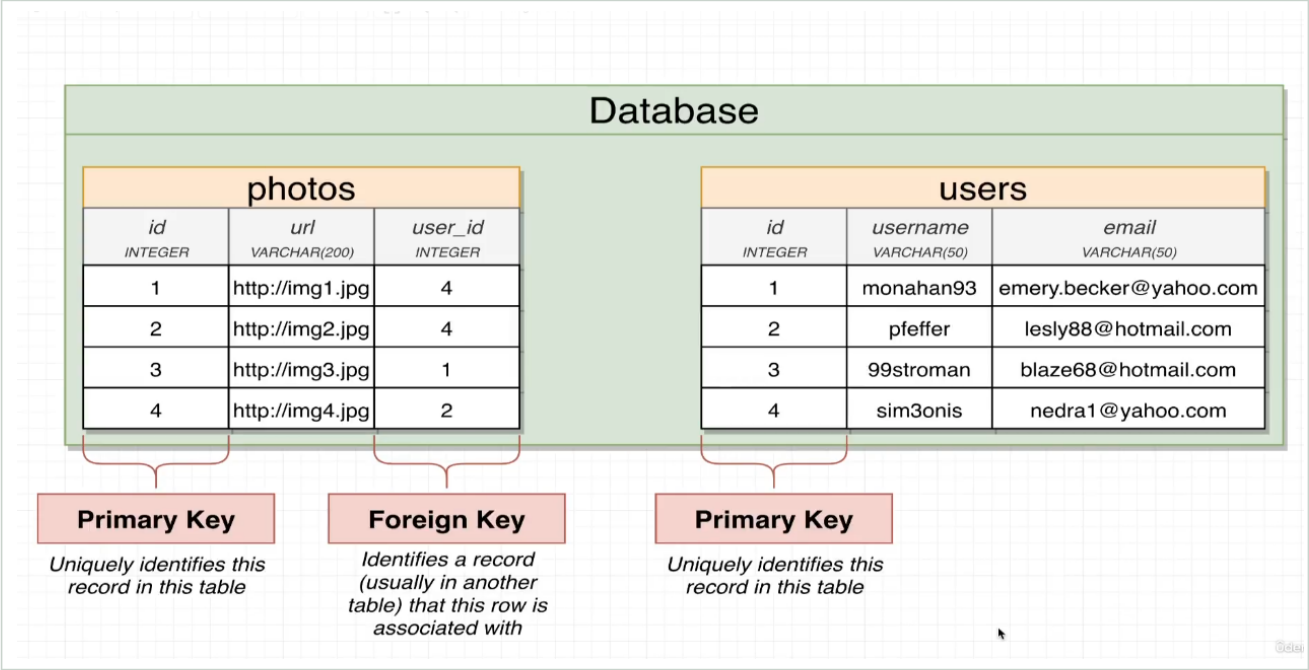


Figure 3 — photos and users tables with primary keys and a foreign key

2. Primary key

- **Every table** we make from now on will have a primary key. (Until now, our tables did not have one.)
- The goal of a primary key: **uniquely identify one row** in a table.
- Every value in this column is **unique**. Example: the first row in photos has an **id** of 1. No other row in this table will ever have an id of 1.
- If we ask for the photo with id 1, we **always** get exactly that row, even if the table is sorted and the row moves.
- The id is always unique, and it **never changes**. In general, we store **numbers** in it.

> **Primary key:** uniquely identifies this record in this table.

3. Foreign key

To set up a relationship between two records, we use a **foreign key**. This is the important part.

- The goal of a foreign key: relate this record to a record in **another table** (or even the same table).
- A user has many photos. So we give the photos table a foreign key column called **user_id**.
- In user_id we store the **id of the user** this photo belongs to.

> **Foreign key:** identifies a record (usually in another table) that this row is associated with.

4. How the link works

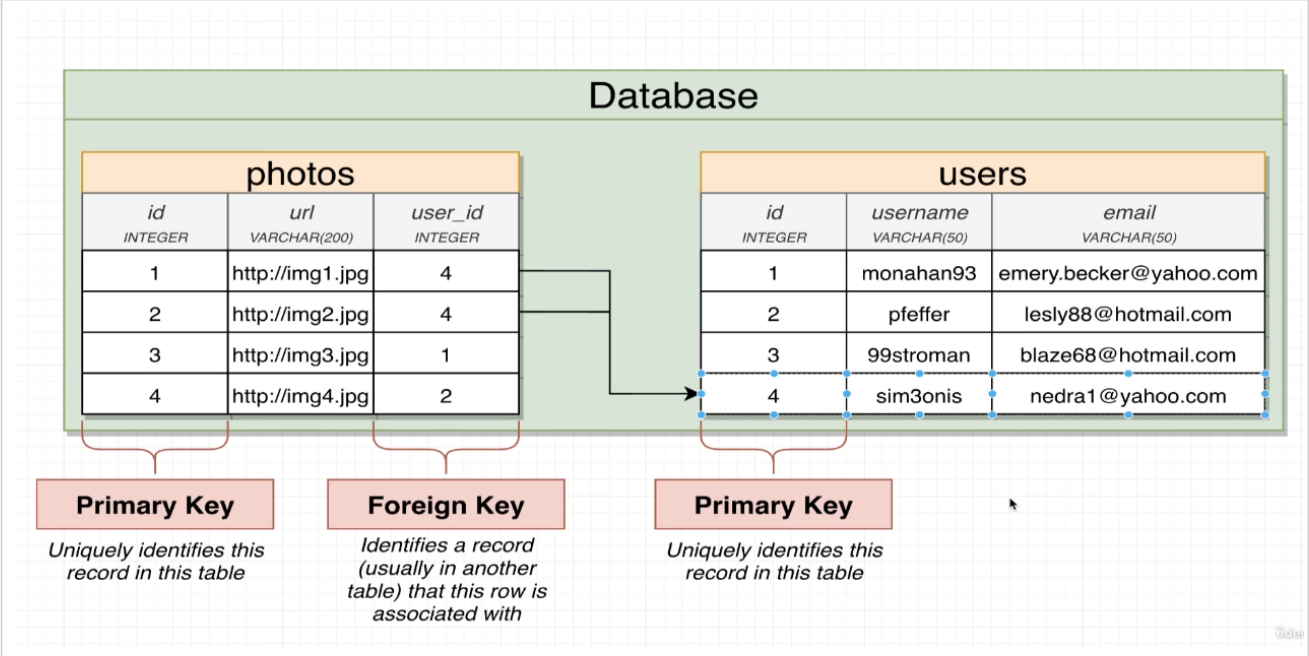


Figure 4 — Photos 1 and 2 have user_id 4, so they belong to user 4 (sim3onis)

- Photos 1 and 2 have **user_id = 4**. So they belong to (or are owned by, or are related to) the user with id 4.
- The users table also has a primary key column called **id**. We look up the user with id 4: **sim3onis**.
- Now these two photos are related to user 4. This is how we build a **one-to-many** and a **many-to-one** relationship. (Remember: they are the same, seen from different sides.)

Finding the photos of a user

We can look at the user_id values in the photos table. "Give me all the photos with a user_id of 4" gives us all the photos of that user.

User (users table)	Look in photos.user_id for	Photos found
id 4 — sim3onis	4	Photo 1, Photo 2
id 1 — monahan93	1	Photo 3
id 2 — pfeffer	2	Photo 4

5. Summary

Idea	Remember
Primary key	Every table has one. Uniquely identifies a row. Usually a number. Always unique, never changes
Foreign key	A column that stores the id of a related record, usually in another table
photos.user_id	Stores the id of the user who owns the photo
One-to-many / many-to-one	Built with a primary key in one table and a foreign key in the other

This setup can be confusing the first time. **Next lesson:** more examples of setting up and keeping these relationships.

Sources: Lecture transcript and lecture images (Figures 1–4)